MIE 1621 Computational Project Version A Winter 2025

Shuman Zhao 1005843991

Part 1. Financial Optimization Model:

$$\begin{aligned} &\text{maximize} \sum_{i=1}^{n} \mu_i x_i - \frac{\delta}{2} \sum_{i=1}^{n} \sum_{i=1}^{n} \sigma_{ij} x_i x_j \\ &\text{subject to} \\ &\sum_{i=1}^{n} x_i = 1 \\ &x_i \geq 0,\ i = 1, \ldots, n \end{aligned}$$

To solve the optimization problem, the model is transformed to:

$$minimize\ \frac{\delta}{2} x^T Q x - \mu^T x$$

with constraints unchanged

$$A = [1 \quad 1 \quad 1], b = 1, c = [0.1073 \quad 0.0737 \quad 0.0627]^T$$

δ is set equal to 4.0

Predictor-Corrector Primal-Dual Interior Point Method:

Step 0: Obtain initial interior solution $(x^{(0)}, \pi^{(0)}, z^{(0)})$ such that $x^{(0)} > 0, z^{(0)} > 0$. Let k = 0 and ε be some small positive number (tolerance). Go to Step 1.

Formulas for initial point generation:

- First compute the following quantities

$$\bar{\pi} = \left(AA^T\right)^{-1} Ac, \quad \bar{z} = c - A^T \bar{\pi}$$

$$\bar{x} = A^T \left(AA^T\right)^{-1} b$$

$$\delta_x = \max\left(-1.5 * \min\{\bar{x}_i\}, 0\right), \quad \delta_z = \max\left(-1.5 * \min\{\bar{z}_i\}, 0\right)$$

- Then form the quantities

$$\bar{\delta}_x = \delta_x + \left[\left(\bar{x} + \delta_x e\right)^T \left(\bar{z} + \delta_z e\right)\right] \Big/ 2\sum_{i=1}^{n} \left(\bar{z}_i + \delta_z\right)$$

$$\bar{\delta}_z = \delta_z + \left[\left(\bar{x} + \delta_x e\right)^T \left(\bar{z} + \delta_z e\right)\right] \Big/ 2\sum_{i=1}^{n} \left(\bar{x}_i + \delta_x\right)$$

- Then an initial point is generated

$$x_i^{(0)} = \bar{x}_i + \bar{\delta_x}, \ z_i^{(0)} = \bar{z}_i + \bar{\delta}_z, \ \forall i = 1, \ldots, n$$

$$\pi^{(0)} = \bar{\pi}$$

The generated initial point is (0.5, 0.5, 0.5), we start with an infeasible point.

Step 1: Solve

$$\begin{bmatrix} \delta Q & A^T & I \\ A & 0 & 0 \\ Z^{(k)} & 0 & X^{(k)} \end{bmatrix} \begin{bmatrix} d_x^{aff} \\ d_\pi^{aff} \\ d_z^{aff} \end{bmatrix} = \begin{bmatrix} -r_d^{(k)} \\ -r_p^{(k)} \\ -X^{(k)}Z^{(k)}e \end{bmatrix}$$

For $\begin{bmatrix} d_x^{aff} \\ d_\pi^{aff} \\ d_z^{aff} \end{bmatrix}$. Go to Step 2.

Formulas to compute $r_d^{(k)}$ and $r_p^{(k)}$:

$$r_p^{(k)} = Ax^{(k)} - b \ \ (primal\ residuals)$$

$$r_d^{(k)} = \delta Q x + A^T \pi^{(k)} + z^{(k)} - c \ \ (dual\ residuals)$$

Step 2: Compute $\alpha_x^{aff}, \alpha_z^{aff}, y^{(k)} and\ y_{aff}^{(k)}$, let $\tau^{(k)} = \left(\frac{y_{aff}^{(k)}}{y^{(k)}}\right)^3$, and solve for $\begin{bmatrix} d_x \\ d_\pi \\ d_z \end{bmatrix}$ in

$$\begin{bmatrix} \delta Q & A^T & I \\ A & 0 & 0 \\ Z^{(k)} & 0 & X^{(k)} \end{bmatrix} \begin{bmatrix} d_x \\ d_\pi \\ d_z \end{bmatrix} = \begin{bmatrix} -r_d^{(k)} \\ -r_p^{(k)} \\ -X^{(k)}Z^{(k)}e - D_{x^{(k)}}D_{z^{(k)}}e + \tau^{(k)}y^{(k)}e \end{bmatrix}$$

Where $D_{x^{(k)}} = diag\left(d_x^{aff}\right) and\ D_{z^{(k)}} = diag\left(d_z^{aff}\right)$. Go to step 3.

Formulas to compute step lengths are:

- The single step length $\alpha^{aff} = \min\left\{1, \min_{i:\left(d_x^{aff}\right)_i<0} -\frac{x_i}{\left(d_x^{aff}\right)_i}, \min_{i:\left(d_z^{aff}\right)_i<0} -\frac{z_i}{\left(d_z^{aff}\right)_i}\right\}$ is used for d^{aff} in place of $\alpha_x{}^{aff}$ and $\alpha_z{}^{aff}$ in Step 2.

Formulas to compute $y^{(k)} and\ y_{aff}^{(k)}$ are:

$$y^{(k)} = \frac{\left(x^{(k)}\right)^T z^{(k)}}{n}, \quad y_{aff} = \frac{\left(x + \alpha_x^{aff} d_x^{aff}\right)^T \left(z + \alpha_z^{aff} d_z^{aff}\right)}{n}$$

Step 3: Compute $\alpha_x \; and \; \alpha_z$ and let

$$x^{(k+1)} = x^{(k)} + \alpha_x d_x$$

$$\pi^{(k+1)} = \pi^{(k)} + \alpha_z d_\pi$$

$$z^{(k+1)} = z^{(k)} + \alpha_z d_z$$

If the stopping criteria is met i.e.

$$\left\| Ax^{(k+1)} - b \right\| \leq \varepsilon$$

$$\left\| \delta Qx + A^T \pi^{(k+1)} + z^{(k+1)} - c \right\| \leq \varepsilon$$

$$(x^{(k+1)})^T z^{(k+1)} \leq \varepsilon$$

For small tolerance $\varepsilon = 10^{-8}$, then STOP

Else k = k+1 and go to Step 1.

Formulas to Compute $\alpha_x \; and \; \alpha_z$:

- The single step length $\alpha = \min\left\{1, \eta\alpha_x^{\max}, \eta\alpha_z^{\max}\right\}$ is used in place of α_x and α_z in Step 3.

Where η is a dampening parameter such that $\eta \in [0.9, 1]$

Computational Result:

Iteration	x	π	z
1	(0.41401349, 0.48572953, 0.14813146)	0.05092098	(0.01282417, 0.00170833, 0.0153533)
2	(0.46358347, 0.51125945, 0.02902211)	0.04427171	(4.39112508e-03, 8.54166667e-05, 1.86546660e-02)
3	(0.49957826, 0.49917705, 0.00146042)	0.04379079	(3.13076084e-04, 4.27083333e-06, 1.89011853e-02)
4	(5.02112875e-01, 4.97824896e-01,	0.04378542	(1.56773208e-05, 2.13552876e-07, 1.88916747e-02)

	7.30211415e-05)		
5	(5.02239790e-01, 4.97757098e-01, 3.65105707e-06)	0.04378516	(7.83868930e-07, 1.06776452e-08, 1.88911935e-02)
6	(5.02246136e-01, 4.97753709e-01, 1.82552854e-07)	0.04378515	(3.91934469e-08, 5.33882258e-10, 1.88911694e-02)
7	(5.02246453e-01, 4.97753539e-01, 9.12764268e-09)	0.04378515	(1.95967234e-09, 2.66941129e-11, 1.88911682e-02)

Optimal objective value is 0.06718031

Compare the results with the results from using MATLAB quadprog (I use CVXPY in python instead):

Optimal objective value from CVXPY is 0.06635042554986054

Optimal x values from CVXPY are (0.11013737, 0.11726218, 0.77260045)

The objective values are very close, but x values are different a lot. This is because the covariance matrix Q is positive definite but with one eigenvalue approximately equal to 0. This makes the objective surface is very flat in one direction, i.e., close to being degenerate. Because the surface is so flat, small numerical differences or different search paths cause different optima.

The eigenvalues are: 0.0045708, 0.04108504, and 0.11454416

Part 2. Large-scale strictly convex quadratic programming

The x values resulted from my code (IPM) and the CVXPY are all the same now. This is because the code in Part 2 creates more symmetric and well-conditioned structure of covariance matrix, which makes both solvers converge to the same x.

The optimal objective values are also the same for two solvers because of the same x values.

CODE

Part 1
Step 0: Problem Setup and Initial Point Generation

```
import numpy as np

# Problem data
mu = np.array([0.1073, 0.0737, 0.0627])       # Expected returns
Sigma = np.array([                            # Covariance matrix
    [0.02778, 0.00387, 0.00021],
    [0.00387, 0.01112, -0.00020],
    [0.00021, -0.00020, 0.00115]
])
delta = 4.0  # Chosen risk aversion parameter

Q = 4 * Sigma
c = mu.copy()                # As we're minimizing, this is used directly
A = np.ones((1, 3))
b = np.array([1.0])
n = len(mu)

# Initial Point Generation (Slide 40 with corrected deltas)
def initial_point(Q, c, A, b):
    e = np.ones(n)

    # Compute intermediate values
    At = A.T
    inv_AAt = np.linalg.inv(A @ At)
    pi_bar = inv_AAt @ (A @ c)
    z_bar = c - At @ pi_bar
    x_bar = At @ inv_AAt @ b

    # Basic deltas
    delta_x = max(-1.5 * np.min(x_bar), 0)
    delta_z = max(-1.5 * np.min(z_bar), 0)
```

```
    x_tilde = x_bar + delta_x * e
    z_tilde = z_bar + delta_z * e

    # Scalar products for bar-deltas
    numerator = np.dot(x_tilde, z_tilde)
    delta_bar_x = delta_x + numerator / (2 * np.sum(z_tilde))
    delta_bar_z = delta_z + numerator / (2 * np.sum(x_tilde))

    # Final initial points
    x0 = x_bar + delta_bar_x * e
    #x0 = x_bar + delta_bar_x
    z0 = z_bar + delta_bar_z * e
    #z0 = z_bar + delta_bar_z
    pi0 = pi_bar

    return x0, pi0, z0
```

Step 1

Solve Predictor Step (affine direction)

```
def predictor_step_qp(Q, A, b, c, x, pi, z):
    n = len(x)
    m = A.shape[0]
    e = np.ones(n)

    # Residuals
    r_d = Q @ x + A.T @ pi + z - c
    r_p = A @ x - b
    r_c = x * z  # Complementarity

    # Diagonal matrices
    X = np.diag(x)
    Z = np.diag(z)

    # Build KKT matrix (NOTE the -Q)
    KKT = np.block([
        [Q, A.T, np.eye(n)],
        [A, np.zeros((m, m)), np.zeros((m, n))],
        [Z, np.zeros((n, m)), X]
    ])

    rhs = -np.concatenate([r_d, r_p, r_c])

    # Solve system
    sol = np.linalg.solve(KKT, rhs)

    dx_aff = sol[:n]
```

```
    dpi_aff = sol[n:n + m]
    dz_aff = sol[n + m:]

    return dx_aff, dpi_aff, dz_aff
```

Step 2

Compute step lengths and centering parameter τ

```
def step_length_and_tau_single(x, z, dx_aff, dz_aff):
    n = len(x)

    # Min step to stay positive in x and z
    alpha_aff_x = min(
        [-x[i] / dx_aff[i] for i in range(n) if dx_aff[i] < 0],
        default=1.0
    )
    alpha_aff_z = min(
        [-z[i] / dz_aff[i] for i in range(n) if dz_aff[i] < 0],
        default=1.0
    )

    alpha_aff = min(1.0, alpha_aff_x, alpha_aff_z)

    # Current duality gap
    y = np.dot(x, z) / n

    # Affine step duality gap
    x_aff = x + alpha_aff * dx_aff
    z_aff = z + alpha_aff * dz_aff
    y_aff = np.dot(x_aff, z_aff) / n

    # Centering parameter
    tau = (y_aff / y) ** 3

    return alpha_aff, tau, y, y_aff
```

Solve for corrector step

```
def corrector_step_qp(Q, A, b, c, x, pi, z, dx_aff, dz_aff, tau, y):
    n = len(x)
    m = A.shape[0]
    e = np.ones(n)

    # Residuals
    r_d = Q @ x + A.T @ pi + z - c
    r_p = A @ x - b
```

```
    # Diagonal matrices
    X = np.diag(x)
    Z = np.diag(z)
    Dx = np.diag(dx_aff)
    Dz = np.diag(dz_aff)

    DxDz_e = Dx @ Dz @ e
    rc = x * z + DxDz_e - tau * y * e  # add two D back

    rhs = -np.concatenate([r_d, r_p, rc])

    KKT = np.block([
        [Q, A.T, np.eye(n)],
        [A, np.zeros((m, m)), np.zeros((m, n))],
        [Z, np.zeros((n, m)), X]
    ])

    sol = np.linalg.solve(KKT, rhs)

    dx_corr = sol[:n]
    dpi_corr = sol[n:n + m]
    dz_corr = sol[n + m:]

    return dx_corr, dpi_corr, dz_corr
```

Step 3

computing the final step size with dampening ($\eta \in [0.9, 1]$) and updating variables

```
def step_and_update(x, pi, z, dx, dpi, dz, eta=0.95):
    n = len(x)

    # Compute individual maximum steps
    alpha_x_max = min([-x[i] / dx[i] for i in range(n) if dx[i] < 0],
default=1.0)
    alpha_z_max = min([-z[i] / dz[i] for i in range(n) if dz[i] < 0],
default=1.0)

    # Multiply η inside min as per the step length formulas
    alpha = min(1.0, eta * alpha_x_max, eta * alpha_z_max)

    # Update variables
    x_new = x + alpha * dx
    pi_new = pi + alpha * dpi
    z_new = z + alpha * dz

    return x_new, pi_new, z_new, alpha
```

looping using def functions above

```
def solve_qp_primal_dual(Q, c, A, b, tol=1e-8, max_iter=1000, eta=0.95,
verbose=True):
    """
    Solves the convex quadratic program:
        minimize   2xᵗQx - cᵗx  => set risk aversion = 4
        subject to Ax = b, x ≥ 0

    using the Predictor-Corrector Primal-Dual Interior Point Method
(Slides 31-40).

    Parameters:
        Q, c, A, b: QP problem data
        tol: convergence tolerance
        max_iter: maximum number of iterations
        eta: dampening factor for step length (e.g., 0.95)
        verbose: whether to print iteration log

    Returns:
        x, pi, z: optimal primal and dual variables
        history: list of iterates (x, pi, z, primal_residual,
dual_residual, complementarity, alpha)
    """

    # Step 0: Generate initial feasible interior point (x > 0, z > 0, Ax =
b, ATπ + z = c)
    x0, pi0, z0 = initial_point(Q, c, A, b)
    x = x0
    pi = pi0
    z = z0

    print("initial point: ", x)

    history = []

    # Main iteration loop
    for k in range(max_iter):
        # Step 1: Predictor step (compute affine directions dx_aff,
dpi_aff, dz_aff)
        dx_aff, dpi_aff, dz_aff = predictor_step_qp(Q, A, b, c, x, pi, z)

        # Step 2: Compute single affine step length and centering
parameter tau
```

```
        alpha_aff, tau, y, y_aff = step_length_and_tau_single(x, z,
dx_aff, dz_aff)

        # Step 2: Corrector step (with second-order correction using DxDz
term)
        dx, dpi, dz = corrector_step_qp(Q, A, b, c, x, pi, z, dx_aff,
dz_aff, tau, y)

        # Step 3: Compute final step length (dampened) and update
variables
        x, pi, z, alpha = step_and_update(x, pi, z, dx, dpi, dz, eta)

        # Step 3: Compute stopping criteria quantities
        primal_res = np.linalg.norm(A @ x - b)                        # ||Ax
- b||
        dual_res = np.linalg.norm(Q @ x + A.T @ pi + z - c)             #
||Qx + Aᵗπ + z - c||
        comp = np.dot(x, z)                                           # xᵗz
(complementarity)

        # Save iteration history for later analysis
        history.append((x.copy(), pi.copy(), z.copy(), primal_res,
dual_res, comp, alpha))

        # Optionally print iteration log
        if verbose:
            print(f"Iter {k}:")
            print(f"  ||Ax - b|| = {primal_res:.2e}")
            print(f"  ||Qx + Aᵗπ + z - c|| = {dual_res:.2e}")
            print(f"  xᵗz = {comp:.2e}")
            print(f"  α = {alpha:.4f}")
            print(f"  x = {x}")
            print(f"  π = {pi}")
            print(f"  z = {z}")

        # Check stopping condition: all residuals and complementarity are
small
        if primal_res < tol and dual_res < tol and comp < tol:
            break

    return x, pi, z, history
```

Executing defined functions to find optimal result

```
x_star, pi_star, z_star, iterates = solve_qp_primal_dual(Q, c, A, b)
```

```
# Compute and print the true (maximization) objective from the original
problem
print("Iter | Expected Return | Variance       | Objective Value")
print("-------------------------------------------------------------")

for i, (x, _, _, _, _, _, _) in enumerate(iterates):
    expected_return = mu @ x
    variance = x @ Sigma @ x
    objective = expected_return - 0.5 * delta * variance  # original
maximization form
    print(f"{i+1:4d} | {expected_return:16.8f} | {variance:13.8f} |
{objective:16.8f}")
```

Compare with function solution

```
import cvxpy as cp
import numpy as np

def solve_with_cvxpy(Q, c, A, b):
    n = len(c)
    x = cp.Variable(n)

    objective = cp.Minimize((2) * cp.quad_form(x, Q) - c @ x)
    constraints = [A @ x == b, x >= 0]
    problem = cp.Problem(objective, constraints)
    problem.solve()

    return x.value, problem.value

x_cvxpy, obj_cvxpy = solve_with_cvxpy(Q, c, A, b)
print("x from cvxpy (quadprog equivalent):", x_cvxpy)
print("Objective value (max form):", mu @ x_cvxpy - 2 * x_cvxpy @ Sigma @
x_cvxpy)
```

reason of the difference: Q is positive definite but with one eigenvalue approximately equal to 0. This makes the objective surface is very flat in one direction, i.e., close to being degenerate. Because the surface is so flat, small numerical differences or different search paths cause different optima.

```
np.linalg.eigvalsh(Q)
```

Part 2

```
def initial_point(Q, c, A, b):
    number = c.shape[0]
    e = np.ones(number)
    A = A.reshape(1, -1)
    At = A.T
    AAt = A @ At

    pi_bar = np.linalg.solve(AAt, A @ c)
    z_bar = c - At @ pi_bar
    x_bar = At @ np.linalg.solve(AAt, b)
```

```
    delta_x = max(-1.5 * np.min(x_bar), 0)
    delta_z = max(-1.5 * np.min(z_bar), 0)

    epsilon = 1e-4  # ensure strict positivity
    x_tilde = np.maximum(x_bar + delta_x * e, epsilon)
    z_tilde = np.maximum(z_bar + delta_z * e, epsilon)

    numerator = np.dot(x_tilde, z_tilde)
    denom_z = np.sum(z_tilde)
    denom_x = np.sum(x_tilde)

    delta_bar_x = delta_x + numerator / (2 * denom_z)
    delta_bar_z = delta_z + numerator / (2 * denom_x)

    x0 = x_bar + delta_bar_x * e
    z0 = z_bar + delta_bar_z * e
    pi0 = pi_bar

    return x0, pi0, z0
```

Generate random covariance matrix

```
def generate_qp_instance(n, seed=None):
    if seed is not None:
        np.random.seed(seed)

    mu = np.ones(n)  # fixed
    A = np.random.randn(n, n)
    Sigma = A + A.T + n * np.eye(n)  # mimic MATLAB code to ensure
positive definite

    Q = 4 * Sigma
    c = mu.copy()
    A_eq = np.ones((1, n))
    b_eq = np.array([1.0])

    return Q, c, A_eq, b_eq, mu, Sigma
```

test generated covariance matrix

```
def compare_solvers(n, seed=0):
    Q, c, A, b, mu, Sigma = generate_qp_instance(n, seed)

    # Solve with your IPM
    x_ipm, pi_ipm, z_ipm, iters_ipm = solve_qp_primal_dual(Q, c, A, b)
```

```
    # Solve with cvxpy
    x_cvx, obj_cvx = solve_with_cvxpy(Q, c, A, b)

    # Compute objective from IPM
    expected_return = mu @ x_ipm
    variance = x_ipm @ Sigma @ x_ipm
    obj_ipm = expected_return - 0.5 * delta * variance  # original
maximization form

    print(f"n = {n}")
    print(f"IPM Objective:    {obj_ipm:.6f}")
    #print(f"CVXPY Objective:  {obj_cvx:.6f}")
    print("CVXPY Objective: ", mu @ x_cvx - 2 * x_cvx @ Sigma @ x_cvx)

    print("x from IPM:", x_ipm)
    print("x from cvxpy (quadprog equivalent):", x_cvx)
```

Run for different size covariance matrix

```
compare_solvers(5)
compare_solvers(10)
compare_solvers(20)
compare_solvers(50)
compare_solvers(100)
compare_solvers(1000)
compare_solvers(2000)
```